

Veil Memory Tuner — Codex Handoff Brief

Project Goal

Build a standalone Roblox prototype for the **Veil Memory Tuner** that can later become a modular subsystem inside the larger **Veil Skiff** co-op game.

The tuner should be designed as a reusable station/minigame system. The later skiff game should not need to know the internal details of selection, input, thread visuals, or action resolution. It should only receive clean gameplay results such as:

- Thread stabilized
- Static cleansed
- Dissonance harmonized
- Overload vented
- Memory progress increased
- Skiff system repaired or protected
- Backlash triggered from a wrong action

The current design favors a simple, readable action system:

The player targets one or more Veil threads, then applies one of four universal tuner actions.

This replaces an earlier slider/dial concept with a faster, more Roblox-friendly thread triage game.

Core Gameplay Concept

The player operates a Veil tuner console that displays several living memory threads. Threads may develop visible problems. The player must quickly identify the problem, target the affected thread or threads, and apply the correct tuner action.

The intended feel is:

A memory medic maintaining unstable living signal threads under pressure.

The core loop is:

1. Threads are visible in the tuner interface or physical station view.
2. One or more threads develop a visible problem.
3. The player targets the relevant thread or threads.
4. The player applies one of four actions.
5. Correct action clears or improves the problem.
6. Wrong action causes instability, damage, wasted time, or backlash.

7. Successful tuning advances memory reconstruction or protects a skiff system.

Four Universal Tuner Actions

The entire core game should use exactly four tuner actions so it maps cleanly to standard Xbox, PlayStation, and Switch controllers.

1. Cleanse

Purpose: remove interference.

Use when a thread is:

- Flickering with static
- Covered in noise particles
- Crackling
- Turning muddy or corrupted
- Producing false pulses

Example problems:

- Static
- Corruption
- Hail impact noise
- Frost buildup, if designed as contamination

2. Anchor

Purpose: stabilize position and prevent drift.

Use when a thread is:

- Pulling away from the bundle
- Wobbling heavily
- Stretching thin
- Slipping or sliding
- Being pushed by wind
- About to snap

Example problems:

- Drift
- Wind shear
- Fraying
- Skiff control instability

- Frost slip, if designed as loss of traction

3. Harmonize

Purpose: restore rhythm, resonance, or pattern alignment.

Use when a thread is:

- Pulsing off-beat
- Wrong color or tone
- Clashing with nearby threads
- Split between memory patterns
- Out of sync with a partner thread

Example problems:

- Dissonance
- Echo mismatch
- Phase clash
- Memory sync failure

4. Vent

Purpose: safely release excess pressure or energy.

Use when a thread is:

- Too bright
- Swollen
- Shaking violently
- Carrying a fast dangerous surge
- About to burst

Example problems:

- Overload
- Backlash buildup
- Heat/pressure surge
- Energy jam

Controller Layout

Use the four shoulder/trigger inputs as the four tuner actions.

Suggested default mapping:

Physical Input	Xbox	PlayStation	Switch	Action
Left top bumper	LB	L1	L	Cleanse
Left bottom trigger	LT	L2	ZL	Anchor
Right top bumper	RB	R1	R	Harmonize
Right bottom trigger	RT	R2	ZR	Vent

Reasoning:

- Left side = maintenance and defensive stabilization.
- Right side = resonance and energy management.
- Top inputs feel lighter and faster.
- Bottom inputs feel heavier or more forceful.

Selection Model

The tuner uses two layers of targeting:

1. Temporary Focus

A thread is temporarily targeted only while the player is actively pointing at it, touching it, aiming at it, or holding its key.

Player-facing term: **in focus**.

2. Locked Selection

A thread remains targeted after the player stops pointing at it.

Player-facing term: **marked**.

Effective Targeting Rule

Actions apply to the union of focused and marked threads.

Effective targets = focused threads + marked threads

This allows simple beginner play:

Point at a thread and press an action.

And advanced play:

Mark several threads, hover another thread, then apply one action to the whole target set.

Input Schemes

The game should support multiple input modes while feeding the same internal selection and action systems.

Mouse + Keyboard

Input	Behavior
Hover thread	Temporarily focus thread
Left click thread	Toggle marked state
Left click drag	Paint-toggle marked state while passing over threads
Right click empty/center	Clear all marks by default; configurable to smart select/deselect all
Q/W/E/R	Apply the four tuner actions

Suggested keyboard action mapping:

Key	Action
Q	Cleanse
W	Anchor
E	Harmonize
R	Vent

Pure Mouse

Pure mouse mode allows the player to play without keyboard action keys.

Input	Behavior
Hover thread	Temporarily focus thread
Left click thread	Toggle marked state
Left click drag	Paint-toggle marked state
Right click empty/center	Clear all marks by default; configurable

Input	Behavior
Scroll wheel	Cycle current tuner action
Middle click / scroll click	Apply current selected action

The UI should clearly show the currently selected action near the cursor or in a small action selector.

Suggested scroll order:

Cleanse → Anchor → Harmonize → Vent → Cleanse

Accessibility alternatives should later include:

- Side mouse button applies current action
- Double click applies current action
- On-screen button applies current action

Controller

Input	Behavior
Aim stick toward thread	Temporarily focus aimed thread
Shoulder/trigger buttons	Apply four tuner actions
Hold stick button while aiming	Paint-toggle marks as aim passes over threads
Press stick button while centered	Smart all command

Smart all command default:

- If any threads are marked: clear all marks.
- If no threads are marked: mark all active threads.

Optional setting:

- Always toggle all marks instead.

Touchscreen

Input	Behavior
Touch thread	Temporarily focus that thread
Drag finger across threads	Temporarily focus thread currently under finger
Multi-touch	Multiple temporary focused threads

Input	Behavior
Tap thread	Toggle marked state
Center button	Smart mark all / clear marks
On-screen action buttons	Apply four tuner actions

Touch does not need drag-to-lock by default because multi-touch already supports multiple temporary focused threads.

Keyboard-Only

Keyboard users can target threads using thread keys.

Suggested default:

Key	Behavior
1-6	Focus corresponding thread while held
Configurable Mark Modifier	If held when a thread key is released, the thread stays marked
Space	Smart mark all / clear marks
Q/W/E/R	Apply four tuner actions

The Mark Modifier should be configurable. Possible defaults:

- Tab
- Shift
- Ctrl
- Enter

Recommended MVP default: **Tab** or **Shift**, but expose as a configurable option.

Keyboard-only behavior:

```

Thread key down:
    add thread to focused set

Thread key up:
    if mark modifier is held:
        remove thread from focused set
        add thread to marked set
    else:
        remove thread from focused set

```

```
Space:
  if any thread is marked:
    clear all marks
  else:
    mark all active threads
```

Optional advanced behavior:

```
Mark Modifier + thread key tap:
  toggle marked state for that thread
```

Visual Selection States

Threads need distinct visuals for focus and marking.

Stable / Untargeted

- Normal glow
- Normal pulse
- No outline

Focused

Temporary targeting.

Suggested visuals:

- Soft outline
- Slight brightening
- Thread subtly lifts or thickens
- Reticle or cursor highlight

Marked

Persistent targeting.

Suggested visuals:

- Stronger persistent outline
- Small glyph/knot/marker attached to thread
- Slightly stronger glow

Focused + Marked

Both temporary and persistent targeting.

Suggested visuals:

- Brightest outline
- Active glyph pulse
- Stronger audio cue
- Thread visibly rises or intensifies

MVP Thread Problems

Start with one clear problem for each of the four actions.

Problem	Visual Behavior	Correct Action	Failure If Ignored
Static	Flickering, sparks, noise crawling along thread	Cleanse	Memory quality drops, nearby noise spreads
Drift	Thread bends/pulls away from bundle	Anchor	Thread frays, breaks, or causes steering instability later
Dissonance	Thread pulses off-beat or wrong color	Harmonize	Memory reconstruction stalls or nearby threads desync
Overload	Thread swells, glows too bright, dangerous pulse races through	Vent	Backlash burst, damage, instability spike

Early levels can spawn only one problem on one thread at a time. Later levels can spawn multiple simultaneous problems.

Difficulty Progression

Stage 1: Live Focus Only

Teach:

Point at the affected thread and press the correct action.

No locked selection needed.

Design:

- One problem at a time
- One affected thread at a time
- Generous response windows
- Clear visual cues

Stage 2: Faster Single-Thread Problems

Teach speed and recognition.

Design:

- Still one affected thread at a time
- Faster pulses
- Shorter response windows
- More frequent problems

Stage 3: Locked Selection

Teach marking.

Design:

- Multiple threads can need the same action
- Player can mark them, then apply one action

Stage 4: Multi-Thread Mechanics

Teach bundle interactions.

Examples:

- Harmonize two or more dissonant threads together
- Anchor a drifting group
- Cleanse a contaminated cluster
- Vent an overloaded bundle

Stage 5: Skiff-Style Pressure

Add hazards and consequences.

Examples:

- Wind causes drift spikes
- Frost causes input delay or slippery thread behavior

- Hail creates false pulses or impact noise
- Corruption spreads between nearby threads
- Overload backlash damages skiff systems

Core Architecture Recommendation

The design should be input-agnostic and modular.

Suggested Roblox structure:

```
ReplicatedStorage
  Shared
    TunerTypes.lua
    TunerConfig.lua
    SelectionState.lua
    TunerActionTypes.lua
    ThreadProblemTypes.lua
    TunerResultTypes.lua

ServerScriptService
  Services
    TunerChallengeService.lua
    ThreadProblemService.lua
    TunerActionResolver.lua
    TunerConsequenceService.lua
    MemoryRewardService.lua

StarterPlayer
  StarterPlayerScripts
    TunerClientController.lua
  InputAdapters
    MouseInputAdapter.lua
    KeyboardInputAdapter.lua
    ControllerInputAdapter.lua
    TouchInputAdapter.lua

StarterGui
  TunerGui
    ThreadView
    ActionDisplay
    StabilityMeter
    MemoryProgressMeter
    TouchActionButtons
```

```
Workspace
  TunerStation
    ConsoleModel
    ThreadBundleVisual
    ProximityPrompt
```

Module Responsibilities

SelectionState

Tracks focused and marked threads.

Example shape:

```
SelectionState = {
  focused = {
    Thread_1 = true,
    Thread_3 = true,
  },
  marked = {
    Thread_2 = true,
  }
}
```

Core functions:

```
FocusThread(threadId)
UnfocusThread(threadId)
ToggleMarkThread(threadId)
MarkThread(threadId)
UnmarkThread(threadId)
ClearMarks()
MarkAll(activeThreadIds)
SmartAll(activeThreadIds)
GetEffectiveTargets()
```

`GetEffectiveTargets()` returns the union of focused and marked threads.

Input Adapters

Each platform-specific input adapter should update `SelectionState` and call the action request function.

Input adapters should not resolve gameplay rules.

They only translate user input into generic commands:

```
SelectionState:FocusThread(threadId)
SelectionState:ToggleMarkThread(threadId)
RequestTunerAction(actionType)
```

TunerActionResolver

Receives action requests and determines success/failure.

Example request:

```
{
  playerId = player.UserId,
  actionType = "Cleanse",
  targetThreads = { "Thread_2", "Thread_4" },
  timestamp = os.clock()
}
```

Example result:

```
{
  actionType = "Cleanse",
  targetThreads = { "Thread_2", "Thread_4" },
  successes = {
    Thread_2 = true,
    Thread_4 = false,
  },
  stabilityChange = 8,
  integrityChange = -2,
  memoryProgressChange = 3,
  backlash = false
}
```

ThreadProblemService

Spawns, clears, and escalates thread problems.

Core functions:

```
SpawnProblem(threadId, problemType)
ClearProblem(threadId)
EscalateProblem(threadId)
GetProblem(threadId)
```

TunerChallengeService

Owns the active challenge state.

Responsibilities:

- Start challenge
- Stop challenge
- Track timer
- Track active threads
- Track global stability
- Track memory progress
- Dispatch problem spawns
- End with success/failure summary

Potential API:

```
StartTuningChallenge(player, challengeConfig)
CancelTuningChallenge(player, reason)
GetChallengeState(player)
OnTunerChallengeComplete(callback)
```

TunerConsequenceService

Converts tuner results into standalone or skiff consequences.

Standalone examples:

- Increase score
- Advance memory progress
- Damage thread integrity
- Trigger backlash visual
- Unlock memory fragment

Future skiff examples:

- Cleansed static improves weapon accuracy
 - Anchored drift improves steering stability
 - Vented overload protects shields
 - Harmonized memory reveals map/path/story data
-

Data Types

Thread Model

```
Thread = {  
  id = "Thread_1",  
  status = "Stable",  
  problem = nil,  
  integrity = 100,  
  pulseSpeed = 1.0,  
  visualIntensity = 1.0,  
  isActive = true,  
  isResolved = false,  
  isBroken = false,  
}
```

Possible statuses:

```
Stable  
Static  
Drifting  
Dissonant  
Overloaded  
Frozen  
Frayed  
Corrupted  
Resolved  
Broken
```

Problem Definition

```
ProblemDefinition = {  
  id = "Static",  
  correctAction = "Cleanse",  
}
```

```

    baseDuration = 6.0,
    escalationTime = 4.0,
    damageOnExpire = 10,
    stabilityPenalty = 5,
    visualCue = "StaticParticles",
    soundCue = "StaticCrackle",
}

```

Action Definition

```

ActionDefinition = {
    id = "Cleanse",
    displayName = "Cleanse",
    cooldown = 0.25,
    maxTargets = nil,
    icon = "CleanseIcon",
    soundCue = "CleansePulse",
}

```

Challenge Config

```

ChallengeConfig = {
    threadCount = 4,
    duration = 60,
    targetMemoryProgress = 100,
    startingStability = 100,
    problemSpawnInterval = 4.0,
    maxSimultaneousProblems = 1,
    allowedProblems = { "Static", "Drift", "Dissonance", "Overload" },
    allowMarkedSelection = false,
    difficulty = 1,
    context = {
        mode = "Standalone",
        targetSystem = "MemoryCore",
    }
}

```

Future skiff context example:

```

context = {
    mode = "Skiff",
    targetSystem = "EngineThreads",
}

```



```
    hazard = "VeilFrost",  
}
```

Action Resolution Rules

MVP rule:

```
If selected thread has a problem and action matches the problem's correct  
action:  
    clear problem  
    increase stability or memory progress  
Else:  
    apply a small penalty
```

MVP should avoid complex partial scoring at first.

Potential first version:

```
if thread.problem and ProblemDefinitions[thread.problem].correctAction ==  
actionType then  
    success = true  
    ClearProblem(thread.id)  
    memoryProgress += 5  
    stability += 2  
else  
    success = false  
    stability -= 3  
    thread.integrity -= 2  
end
```

Later additions:

- Timing windows
- Combo bonuses
- Multi-target action falloff
- Problem severity
- Chain reactions
- Wrong action-specific penalties
- Cooldowns
- Skiff system-specific outcomes

MVP Build Milestones

Milestone 1: Static Thread Display

Goal: show 4–6 Veil threads.

Deliverables:

- Tuner station or UI scene
- Thread visuals
- Basic focused and marked states
- Placeholder art acceptable

Milestone 2: Mouse Hover Targeting + Four Actions

Goal: prove simplest core loop.

Deliverables:

- Hover focuses thread
- Q/W/E/R apply Cleanse/Anchor/Harmonize/Vent
- One problem type can spawn
- Correct action clears problem

Milestone 3: Four Problem Types

Goal: one problem per action.

Deliverables:

- Static → Cleanse
- Drift → Anchor
- Dissonance → Harmonize
- Overload → Vent
- Visual distinction for each problem

Milestone 4: Locked Selection

Goal: allow multi-thread targeting.

Deliverables:

- Mouse click toggles marked state
- Marked + focused threads are both targeted
- Clear all marks command

Milestone 5: Challenge Loop

Goal: make it a playable minigame.

Deliverables:

- Timed challenge
- Problem spawning
- Stability meter
- Memory progress meter
- Success/failure condition
- Basic rewards/results summary

Milestone 6: Controller Support

Goal: prove console-compatible control scheme.

Deliverables:

- Stick aims/focuses thread
- Shoulder/trigger actions work
- Stick button marking works
- Centered stick smart all command works

Milestone 7: Pure Mouse + Touch Support

Goal: broaden input support.

Deliverables:

- Scroll wheel cycles current action
- Middle click applies current action
- Touch focus/tap/multitouch action buttons

Milestone 8: Skiff Integration Interface

Goal: make the tuner reusable later.

Deliverables:

- Public challenge API
- Clean result events
- Context-based consequences
- No hard dependency on standalone game rewards

Required Placeholder Assets for MVP

The next stage is asset planning. For the prototype, placeholder assets are acceptable, but the system should be built so final assets can be swapped in.

Models / 3D

- Tuner console model
- Thread bundle holder/frame
- Optional floating crystal/memory core
- Optional skiff-compatible tuner station shell

Thread Visuals

- Stable glowing thread
- Focused thread highlight
- Marked thread highlight/glyph
- Static problem effect
- Drift problem effect
- Dissonance problem effect
- Overload problem effect
- Success pulse
- Wrong action backlash pulse

UI Graphics

- Cleanse icon
- Anchor icon
- Harmonize icon
- Vent icon
- Current action selector for pure mouse mode
- Stability meter
- Memory progress meter
- Thread status markers
- Touchscreen action buttons

Sounds

- Thread ambient hum
- Focus hover tone
- Mark/unmark click or glyph sound
- Cleanse action sound
- Anchor action sound
- Harmonize action sound
- Vent action sound
- Correct action success sound

- Wrong action error sound
- Backlash burst
- Memory fragment reveal sound
- Challenge start/end sound

Optional Music / Ambience

- Low Veil ambience loop
- Rising tension layer as instability increases
- Soft musical harmony layer as memory progress increases

First Codex Task Recommendation

Start with the MVP code skeleton and mouse + keyboard prototype.

Specific first task:

Build a Roblox tuner prototype with four visible threads. Hovering a thread focuses it. Q/W/E/R apply Cleanse, Anchor, Harmonize, and Vent to the focused thread. Each thread can randomly develop one of four problems, each requiring its matching action. Correct actions clear problems and increase memory progress. Wrong actions reduce stability.

Avoid overbuilding assets in the first pass. Use simple Parts, Beams, BillboardGuIs, and placeholder colors/effects until the gameplay loop works.

Design Rule Going Forward

Every future hazard or mechanic should be expressible through:

1. Which threads are targeted
2. Which of the four actions is applied
3. Whether timing, grouping, or context changes the result

Do not add a fifth core action unless absolutely necessary.

The central design should remain:

Target thread(s), then apply Cleanse, Anchor, Harmonize, or Vent.